

IA

Asistente de Diagnostico de Hardware IA

Documentacion Tecnica Completa

Flask

Angular 21

scikit-learn

Supabase

LinearSVC

TF-IDF

2 836

Ejemplos ML

9

Categorias

15

Marcas

99%

Accuracy

<10ms

Inferencia

Generado: 04/06/2026 08:56 | github.com/Franck147/Machine-Learning-con-Angular

Indice de Contenido

1.	Descripcion del Proyecto	3
2.	Arquitectura del Sistema	4
3.	Motor de Machine Learning	5
4.	Como se Entrena el Modelo	6
5.	Metricas Reales de Rendimiento	7
6.	Categorias de Diagnostico	8
7.	Marcas y Series Soportadas	9
8.	Base de Datos - Supabase	10
9.	API REST - Endpoints	11
10.	Frontend Angular 21	12
11.	Flujo de Datos Completo	13
12.	Feedback Loop y Re-entrenamiento	14
13.	Como Alimentar el Sistema	15
14.	Limitaciones y Mejoras Futuras	16
15.	Como Ejecutar el Proyecto	17

1. Descripción del Proyecto

El Asistente de Diagnóstico de Hardware IA es un sistema web full-stack que permite diagnosticar fallas de hardware de laptops, PCs y tablets usando inteligencia artificial. El usuario describe el síntoma en lenguaje natural y el sistema lo clasifica en una de 9 categorías, devolviendo una solución técnica específica para la marca del equipo.

Problema que resuelve

El diagnóstico de hardware requiere conocimiento especializado. Un usuario no técnico no sabe si 'la pantalla negra' es GPU, energía, BIOS o drivers. Este sistema actúa como primer nivel de triaje técnico, reduciendo el tiempo de diagnóstico de 30 minutos a menos de 1 segundo.

Características principales

- Clasificación ML en tiempo real: texto libre -> 9 categorías en <10ms
- Soluciones específicas por marca: Dell, HP, Lenovo, ASUS, Acer, MSI y 9 marcas más
- Umbral de confianza: si el modelo duda (<45%), pide más información al usuario
- Conversación multi-turno: acumula contexto de mensajes anteriores
- Feedback loop: botones útil/no útil que alimentan logs en Supabase
- 2836 ejemplos de entrenamiento: 336 manuales + 2500 del dataset CSV
- Base de datos persistente: Supabase (PostgreSQL) con 262+ soluciones
- Frontend dark/red: Angular 21 con Angular Material

Tipo de ML usado

Aprendizaje supervisado de clasificación de texto (NLP). El modelo aprende de ejemplos etiquetados (síntoma -> categoría) y generaliza a frases nuevas. No usa redes neuronales ni LLMs. Usa álgebra lineal clásica (SVM) que es más rápido, explicable y no requiere GPU.

2. Arquitectura del Sistema

Diagrama de capas

```
+-----+
|          FRONTEND (Angular 21) -- localhost:4200          |
| Chat UI - Selector Marca/Serie - Feedback - Material Design |
+-----+
| HTTP/REST (/api) |
+-----v-----+
|          BACKEND (Flask 3.1) -- localhost:5000          |
| POST /api/diagnosticar POST /api/feedback GET /health      |
+-----+
|          |          |
+-----v-----+ +-----v-----+
| Modelo ML      | | Supabase (PostgreSQL)                  |
| TF-IDF + LinearSVC | | brands - model_series                |
| 2836 ejemplos  | | catalog_solutions - diagnosis_logs      |
| 9 categorias   | | 262+ soluciones por marca                |
+-----+ +-----+
```

Archivos clave

Archivo	Tecnologia	Funcion
backend/model_logic.py	scikit-learn	Modelo ML: TF-IDF + LinearSVC
backend/app.py	Flask 3.1	API REST: 3 endpoints
backend/import_dataset.py	csv/python	Importador de datasets CSV
backend/extra_training_data.py	Python	2500 ejemplos del CSV
database/schema.sql	PostgreSQL	Schema completo de Supabase
data/dataset_diagnosticos.csv	CSV	2500 diagnosticos reales
frontend/.../chat.component/	Angular 21	Interfaz de chat completa
frontend/.../diagnosis.service.ts	TypeScript	Servicio HTTP al backend

3. Motor de Machine Learning

El pipeline: 2 pasos

El sistema usa un pipeline de dos pasos: TF-IDF para convertir texto en numeros, y LinearSVC para clasificar. Esta combinacion es la mas eficiente para clasificacion de texto corto en espanol.

Paso 1: TF-IDF (Term Frequency - Inverse Document Frequency)

Convierte cada frase en un vector numerico. Asigna pesos segun la frecuencia de cada palabra/bigrama en esa frase versus en todo el dataset. Palabras especificas como 'click click' (disco duro) o 'beep codes' (BIOS) obtienen pesos altos porque son raras en el resto del dataset.

```
Entrada: 'el disco hace click click y no arranca'
```

```
Tokens/bigramas con mayor peso:
```

```
'click click' -> 0.81    (bigrama MUY especifico de HDD)
'hace click'  -> 0.71    (patron caracteristico de disco)
'no arranca'  -> 0.56    (bigrama diagnostico)
'disco'       -> 0.42
```

```
Ignorados (alta frecuencia en todo): 'el', 'y', 'no'
```

```
ngram_range=(1,2): captura unigramas Y bigramas
```

```
sublinear_tf=True: usa log(1+tf) para suavizar frecuencias altas
```

```
strip_accents='unicode': 'diagnostico' = 'diagnostico' (sin tilde)
```

Paso 2: LinearSVC (Support Vector Classifier)

Traza hyperplanos de separacion en el espacio de alta dimension. Para cada par de categorias, aprende el hyperplano optimo que maximiza el margen entre las dos clases. Con 9 categorias usa estrategia One-vs-One (36 clasificadores binarios).

```
class_weight='balanced' -> compensa categorias con menos ejemplos
C=1.0                    -> regularizacion media (balance bias/varianza)
max_iter=2000            -> convergencia garantizada
```

```
CalibratedClassifierCV(cv=3):
```

```
Envuelve LinearSVC para generar probabilidades reales (0-100%).
```

```
Usa Platt scaling para convertir distancias al hyperplano en probs.
```

```
Sin esto: solo +/-distancia, no porcentaje de confianza.
```

Por que LinearSVC y no otros modelos?

Modelo	Accuracy	Velocidad	Memoria	Explic.	Mejor para
LinearSVC *	99%	~2s train	Baja	Alta	Texto corto clasificacion
Naive Bayes	87%	~0.5s	Baja	Alta	Spam, texto simple
Random Forest	93%	~15s	Media	Media	Datos tabulares
BERT fine-tune	~99%	~5min GPU	5GB+	Baja	NLP complejo largo
GPT/Claude	~95%	1-5s/req	Nube	Baja	Generacion de texto

4. Como se Entrena el Modelo

Proceso en codigo

```
# model_logic.py -- Entrenamiento completo

class DiagnosticModel:
    def __init__(self):
        self.pipeline = Pipeline([
            ('tfidf', TfidfVectorizer(
                ngram_range=(1, 2),      # unigramas Y bigramas
                sublinear_tf=True,       # log(1+tf)
                strip_accents='unicode', # normaliza acentos
                lowercase=True,          # todo a minusculas
            )),
            ('clf', CalibratedClassifierCV(
                LinearSVC(C=1.0, max_iter=2000, class_weight='balanced'),
                cv=3, # calibracion interna con 3 folds
            )),
        ])
        self._train()

    def _train(self):
        data = TRAINING_DATA # 336 ejemplos manuales
            + _EXTRA          # 2500 del CSV importado
        texts, labels = zip(*data)
        self.pipeline.fit(texts, labels) # ~2 segundos
```

Fuentes de datos de entrenamiento

Fuente 1: Ejemplos manuales escritos a mano (336 ejemplos)

30 ejemplos por cada una de las 9 categorías = 270 genericos. Mas 66 ejemplos con prefijo de marca: [DELL XPS], [HP OMEN], etc. Cubren los sintomas mas comunes y frases naturales que usan los usuarios.

```
('el disco hace click click y no arranca',      'Almacenamiento'),
('pantalla con lineas verticales y artefactos',  'Video'),
('[ASUS ROG] Armoury Crate causa BSOD',         'Drivers'),
('[LENOVO LEGION] GPU NVIDIA no activa en juegos','Video'),
```

Fuente 2: Dataset CSV importado (2500 ejemplos)

Archivo data/dataset_diagnostics_equipos.csv con 2500 casos reales de 15 marcas. El script import_dataset.py mapea el campo Componente_Fallido a las 9 categorías del modelo y genera los ejemplos con prefijo [MARCA MODELO] automaticamente.

```
COMPONENT_MAP = {
    'Disco Duro/SSD': 'Almacenamiento', 'RAM': 'BIOS',
    'Pantalla': 'Video',                 'GPU': 'Video',
    'Bateria': 'Energia',                 'Fuente de Poder': 'Energia',
```


5. Metricas Reales de Rendimiento

Evaluacion 80/20 (test set = 568 ejemplos nunca vistos)

Categoria	Precision	Recall	F1-Score	Soporte	Estado
Almacenamiento	1.00	0.99	0.99	82	Excelente
Audio	1.00	1.00	1.00	19	Perfecto
BIOS	0.99	0.99	0.99	149	Excelente
Drivers	0.97	1.00	0.98	30	Muy bueno
Energia	0.98	0.97	0.98	62	Muy bueno
Red	0.97	0.93	0.95	30	Bueno
Temperatura	0.98	1.00	0.99	47	Excelente
USB	0.98	1.00	0.99	61	Excelente
Video	0.99	0.98	0.98	88	Excelente

Resumen global

Accuracy total (test set 20%):	99%
F1-Score macro promedio:	0.98
Cross-validation 5-fold:	95.1% (+/-9.5%)
Tiempo de entrenamiento:	~2 segundos (2836 ejemplos)
Tiempo de inferencia:	<10 ms por prediccion
Categoria mas debil:	Red (F1=0.95)
Categoria perfecta:	Audio (F1=1.00)

Comportamiento con entradas ambiguas

Frase descriptiva:	Prediccion	Conf.	Resultado
'disco hace click click'	-> Almacenam.	87%	Correcto
'pantalla con artefactos'	-> Video	97%	Correcto
'wifi desconexion'	-> Red	90%	Correcto
'temperatura 95 grados'	-> Temperatura	95%	Correcto
Frase ambigua/corta:	Prediccion	Conf.	Accion
'pantalla negra al encender'	-> Video	70%	Acepta
'se apaga bajo carga'	-> Energia	81%	Acepta
'no enciende'	-> USB	46%	BAJO UMBRAL -> pregunta
'falla'	-> Drivers	54%	Acepta (sobre umbral)
'problemas'	-> Audio	18%	PIDE ACLARACION

6. Categorías de Diagnostico (9 total)

El modelo clasifica en 9 categorías. Cada una tiene soluciones en Supabase diferenciadas por marca y serie del equipo.

Energia

PSU, batería, fuente, reinicios, no enciende

Solucion: Verificar PSU, voltajes ATX, adaptadores

Video

GPU, pantalla, artefactos, sin senal HDMI/DP

Solucion: Drivers graficos, reinsercion GPU, cables

BIOS

RAM, POST, beep codes, firmware, arranque

Solucion: Reset CMOS, reubicacion RAM, actualizar BIOS

Almacenamiento

HDD, SSD, NVMe, sectores, SMART, no bootea

Solucion: CrystalDiskInfo, chkdsk, reemplazo disco

Red

WiFi, ethernet, drivers red, IP, DNS

Solucion: Reinstalar drivers, netsh winsock reset

Audio

Altavoces, microfono, jack, HDMI audio

Solucion: Drivers Realtek, dispositivo predeterminado

Temperatura

Sobrecalentamiento, ventiladores, throttling

Solucion: Limpieza polvo, pasta termica, curvas fan

USB

Puertos USB, dispositivos no reconocidos

Solucion: Reinstalar controladores USB, hub interno

Drivers

BSOD por driver, Device Manager, Windows

Solucion: DDU, WhoCrashed, sitio oficial fabricante

7. Marcas y Series Soportadas (15 marcas)

El lookup de soluciones tiene 3 niveles de prioridad: brand+serie+categoria > brand+categoria > generica.

Dell	XPS, Inspiron, Latitude, Alienware, OptiPlex	Dell Command Update, Pre-Boot Diagnostics (F12)
HP	Pavilion, Spectre, EliteBook, Omen, Envy	HP Support Assistant, HP PC Hardware Diagnostics (F2)
Lenovo	ThinkPad, IdeaPad, Legion, Yoga, ThinkCentre	Lenovo Vantage, System Update, Boton Novo
ASUS	VivoBook, ZenBook, ROG, TUF, ExpertBook	MyASUS, Armoury Crate, EZ Flash 3
Acer	Aspire, Swift, Nitro, Predator, TravelMate	Acer Care Center, PredatorSense, eRecovery
MSI	GF Series, GE Series, Stealth, Creator	MSI Center, Cooler Boost (Fn+F7)
Apple	MacBook Air, MacBook Pro, Mac mini, iMac	Apple Diagnostics (D al encender)
Samsung	Galaxy Book, Galaxy Tab	Samsung Update, Samsung Diagnostics
Huawei	MateBook, MatePad	PC Manager, Huawei PC Manager Diagnostics
Toshiba	Satellite, Dynabook	Toshiba Service Station, PC Diagnostic Tool
Microsoft	Surface Pro, Surface Laptop, Surface Book	Surface Diagnostic Toolkit, Firmware updates
Xiaomi	Mi Notebook, Pad	Mi PC Manager
Amazon	Fire, Fire HD	Amazon Device Support, Fire OS reset
Gigabyte	AORUS, B Series, Z Series	Gigabyte Control Center, Q-Flash BIOS
Generica	PC Escritorio, Laptop, Gaming PC	Pasos generales universales

8. Base de Datos - Supabase / PostgreSQL

Tablas del sistema

brands

15 marcas con herramienta de soporte y URL oficial

name PK | display_name | support_tool | diagnostic_tool | support_url

model_series

56+ series con tipo (laptop/gaming/desktop/workstation)

id UUID PK | brand_name FK | series | type | description

catalog_solutions

280+ soluciones. brand/series NULL = generica para todos

id UUID PK | category NOT NULL | brand NULL | series NULL | solution_text | hardware_specs JSONB

diagnosis_logs

Cada diagnostico con feedback del usuario

id UUID PK | user_query | predicted_category | accuracy | brand | series | feedback_util BOOL

Estrategia de lookup de soluciones (3 niveles)

```
# app.py -- get_solution_from_supabase(category, brand, series)

# Nivel 1: solucion especifica brand + serie + category
#   Ej: Dell + XPS + Temperatura -> pasta metal liquido para XPS 15

# Nivel 2: solucion de la marca + category
#   Ej: Dell + Temperatura -> Dell Power Manager, thermals generales

# Nivel 3: solucion generica por category (brand=NULL)
#   Ej: Temperatura -> limpiar polvo, pasta termica, ventiladores

# Nivel 4: FALLBACK_SOLUTIONS (hardcoded, funciona sin Supabase)
```

9. API REST - Endpoints

GET /api/health -- Verificar estado del servidor

```
curl http://localhost:5000/api/health
```

Respuesta:

```
{"status": "ok", "model": "DiagnosticModel v2.0",  
  "algoritmo": "TF-IDF + LinearSVC",  
  "categorias": ["Energia", "Video", "BIOS", ...],  
  "umbral_confianza": 0.45}
```

POST /api/ diagnosticar -- Clasificar sintoma

Request:

```
{  
  "mensaje": "pantalla negra GPU no detectada",  
  "marca":   "Dell",           // opcional  
  "serie":   "Alienware",      // opcional  
  "contexto": ["..."]         // opcional (multi-turno)  
}
```

Respuesta exitosa (confianza >= 45%):

```
{  
  "categoria": "Video",  
  "confianza": 0.97,  
  "solucion": "Para problemas de video en Dell: ...",  
  "probabilidades": {"Video": 0.97, "Energia": 0.01, ...},  
  "log_id": "uuid-del-registro",  
  "marca": "Dell", "serie": "Alienware"  
}
```

Respuesta baja confianza (< 45%):

```
{  
  "necesita_mas_info": true,  
  "pregunta": "No pude identificar el problema...",  
  "confianza_maxima": 0.23  
}
```

POST /api/feedback -- Registrar utilidad del diagnostico

Request:

```
{"log_id": "uuid-del-diagnostico", "util": true}
```

Respuesta:

```
{"ok": true}
```

```
# Actualiza diagnosis_logs.feedback_util = true/false
```

```
# Los registros con feedback_util=true se usan para re-entrenar
```

10. Frontend Angular 21

Stack de tecnologías

Angular 21.2	Framework SPA con Standalone Components y Signals
Angular Material 21	Card, Chips, Button, Icon, Progress Spinner
Angular Signals	Estado reactivo: signal(), computed() sin RxJS
RxJS 7.8	HTTP observable y manejo de errores con catchError
TypeScript 5.9	Interfaces discriminadas DiagnosisResult ClarificationResult
SCSS	Variables, anidamiento, tema dark/red, responsive
Proxy Angular	proxy.conf.json: /api -> localhost:5000 en dev

Componentes y signals

```
ChatComponent (chat.component.ts)
  signals:
    messages:      ChatMessage[]    -- historial del chat
    userInput:     string            -- texto del usuario
    isLoading:     boolean           -- esperando respuesta
    selectedBrand: string|null       -- marca seleccionada
    selectedSeries: string|null      -- serie seleccionada
  computed:
    availableSeries: BrandSeries[]   -- series de la marca activa
    canSend: boolean                 -- habilita boton enviar
  methods:
    sendMessage() -- llama API con contexto y marca
    sendFeedback() -- envia util/no-util al backend

DiagnosisService (diagnosis.service.ts)
  diagnose(msg, ctx?, marca?, serie?) -> Observable<DiagnosisResponse>
  sendFeedback(log_id, util)           -> Observable<{ok:boolean}>
```

UI: tema oscuro con rojo

- Fondo #080808 con glow rojo sutil por radial-gradient
- Header degradado rojo oscuro: #1a0000 -> #2d0000
- Stat pills: 2836 ejemplos - 9 categorias - 15 marcas - Supabase
- Burbujas usuario: gradiente rojo #7f0000 -> #c62828
- Burbujas bot: #1a1a1a con borde oscuro
- Barra de confianza: verde(>=80%), naranja(>=60%), rojo(<60%)
- Chips de probabilidad con icono de categoria
- 15 marcas con colores oficiales, series con icono por tipo

11. Flujo de Datos Completo (end-to-end)

```
USUARIO escribe: 'ventilador al maximo y se apaga jugando'
+ selecciona: ASUS ROG
    |
    v
ANGULAR sendMessage()
  DiagnosisService.diagnose('ventilador...', [], 'ASUS', 'ROG')
  POST /api/diagnosticar
  { mensaje: '...', marca: 'ASUS', serie: 'ROG' }
    |
    v
FLASK /api/diagnosticar
  1. Enriquecer: '[ASUS ROG] ventilador al maximo y se apaga jugando'
  2. TF-IDF vectoriza -> vector 15000+ dimensiones
  3. LinearSVC predice -> { category: 'Temperatura', conf: 0.91 }
  4. Confianza 91% >= 45%: NO pide aclaracion
  5. Supabase: SELECT WHERE brand='ASUS' AND series='ROG'
      AND category='Temperatura'
      -> solucion especifica ROG: Armoury Crate Fan Profile...
  6. INSERT diagnosis_logs (query, 'Temperatura', 0.91, 'ASUS', 'ROG')
  7. Retorna: { categoria, confianza, solucion, probs, log_id }
    |
    v
ANGULAR muestra burbuja:
  - Badge 'Temperatura' con barra 91% verde
  - Badge 'ASUS ROG' con icono laptop
  - Solucion especifica ROG
  - Chips de probabilidades con iconos
  - Botones 'Fue util?' (thumbs up/down)
    |
    USUARIO hace click en thumbs up
      v
  sendFeedback(log_id, true)
  POST /api/feedback { log_id, util: true }
  Supabase UPDATE diagnosis_logs SET feedback_util=true
```

12. Feedback Loop y Re-entrenamiento

Ciclo de mejora continua

```
1. Usuario recibe diagnostico -> hace click en 'Fue util'
2. diagnosis_logs.feedback_util = true

3. Script de re-entrenamiento (semanal/manual):
  SELECT user_query, predicted_category FROM diagnosis_logs
  WHERE feedback_util = true
  AND created_at > NOW() - INTERVAL '30 days'

4. model.retrain(nuevos_ejemplos)
   -> self._extra_data.extend(nuevos_ejemplos)
   -> self._train() (~2 segundos)

5. Reiniciar servidor para usar modelo actualizado
```

Fuentes para re-entrenamiento

Fuente	Descripcion	Confiabilidad
Feedback positivo	Diagnosticos donde usuario confirmo 'Si fue util'	Alta -- usuario valido
Alta confianza	Accuracy >= 85% sin feedback negativo	Media -- modelo seguro
Nuevo CSV	Archivos CSV mismo formato del importador	Alta -- datos etiquetados
Claude API	Generar 100+ ejemplos por categoria con IA	Media -- sinteticos

13. Como Alimentar el Sistema con Nuevos Datos

Opcion A: Nuevo CSV (recomendado)

```
# Formato requerido del CSV:
Marca, Modelo, Componente_Fallido, Error_Especifico, Solucion_Recomendada

# Componentes validos (mapeados a categorias ML):
Disco Duro/SSD, RAM, Motherboard, Pantalla, Bateria, Ventilacion,
GPU, Teclado, Puertos, BIOS, Conectividad, Sistema Operativo,
Fuente de Poder, Audio

# Ejecutar importador:
cd backend
venv\Scripts\python import_dataset.py ../data/nuevo_dataset.csv

# Reiniciar backend:
python app.py
```

Opcion B: Generar con Claude API

```
import anthropic
client = anthropic.Anthropic()
response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=2048,
    messages=[{
        'role': 'user',
        'content': 'Genera 50 frases de sintomas de PC en espanol
para la categoria Audio. Frases naturales, una por linea.'
    }]
)
```

Opcion C: Agregar directamente a model_logic.py

```
# Al final de TRAINING_DATA:
('el microfono capta ruido de fondo excesivo', 'Audio'),
('[APPLE MACBOOK] sin sonido tras actualizar OS', 'Audio'),
('[SAMSUNG GALAXY] altavoces no suenan', 'Audio'),
```

14. Limitaciones y Mejoras Futuras

Limitaciones actuales

Frases muy cortas

Frases de 1-2 palabras tienen baja confianza (ej: 'no enciende' -> 46%). El modelo necesita mas contexto descriptivo para operar bien.

Sin clasificacion multi-label

Una frase puede describir 2 problemas (pantalla negra + no enciende = Video + Energia). El modelo solo predice 1 categoria por frase.

Sin persistencia del modelo en disco

El modelo se re-entrena en cada inicio del servidor (~2s). No se guarda con joblib/pickle entre reinicios.

Sin aprendizaje en tiempo real

El feedback loop requiere reinicio manual. No hay actualizacion online del modelo con los datos nuevos mientras el servidor corre.

Solo espanol

Todo el dataset y las soluciones estan en espanol. No hay soporte para otros idiomas.

Roadmap de mejoras

Mejora	Descripcion	Impacto
Persistencia joblib	joblib.dump/load: elimina 2s de re-entrenamiento al inicio	Alto -- performance
Embeddings semanticos	Reemplazar TF-IDF por sentence-transformers multilingue	Alto -- precision
Multi-label	MultiOutputClassifier para predecir 2+ categorias a la vez	Medio -- UX
Auto-retraining cron	Script semanal que consulta Supabase y llama model.retrain()	Alto -- automatizacion
Dashboard analytics	Pagina con estadisticas: categorias frecuentes, tasa feedback	Medio -- visibilidad
Endpoint /api/admin/retrain	Dispara re-entrenamiento desde el frontend sin reiniciar	Alto -- operaciones

15. Como Ejecutar el Proyecto

Requisitos

- Python 3.11+
- Node.js 20+ con npm
- Cuenta Supabase (plan gratuito)
- Git

Instalacion completa (primera vez)

```
# 1. Clonar
git clone https://github.com/Franck147/Machine-Learning-con-Angular
cd Machine-Learning-con-Angular

# 2. Backend
cd backend
python -m venv venv
venv\Scripts\Activate          # Windows
source venv/bin/activate       # Linux/Mac
pip install -r requirements.txt

# 3. Configurar .env
copy .env.example .env
# Editar .env: SUPABASE_URL y SUPABASE_KEY

# 4. Crear tablas en Supabase
# SQL Editor en supabase.com -> pegar database/schema.sql -> Run

# 5. Deshabilitar RLS (para la clave anon)
# ALTER TABLE brands DISABLE ROW LEVEL SECURITY;
# ALTER TABLE model_series DISABLE ROW LEVEL SECURITY;
# ALTER TABLE catalog_solutions DISABLE ROW LEVEL SECURITY;
# ALTER TABLE diagnosis_logs DISABLE ROW LEVEL SECURITY;

# 6. Cargar datos del CSV a Supabase
python import_dataset.py

# 7. Iniciar backend
python app.py    # -> http://localhost:5000

# 8. Frontend
cd ..\frontend
npm install
ng serve         # -> http://localhost:4200
```

Verificacion rapida


```
curl http://localhost:5000/api/health
```


